Course Project Report

南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

CS100 | Engineering Report Writing / 工程报告写作

Modernized Template Smoke Test / 模板冒烟测试

Team Members (2)

Wells *Lead Developer*
wells@example.com
12345678 • Department of Computer Science and Engineering

Teammate *Experiment Design*
teammate@example.com
12340000 • Department of Electronic and Electrical Engineering

CRediT-style Contributions

Wells
Roles: Conceptualization, Software, Writing - Original Draft

Teammate
Roles: Investigation, Validation, Writing - Review and Editing

Contribution Summary

Software Development
Wells implemented the template core refactor and metadata pipeline.

Experiment Design
Teammate organized the report structure and validation scenario.

Instructor: Prof. Example
Semester: Spring 2026

April 5, 2026

Contents

1 Introduction **1**

2 Math **1**

3 Code **2**

4 Code — Extended Languages and Advanced Blocks **2**

4.1 Modern C++ . . . 2
4.2 MATLAB . . . 3
4.3 Enhanced YAML . . . 3
4.4 Long Code Block with Pagination . . . 3
4.5 Diff-Style Code Block . . . 4

5 Table **5**

6 Figure **5**

7 List **5**

8 Grouped Tables **6**

9 Diagram **6**

10 Conclusion **6**

1 Introduction

This file is a minimal regression example for the class refactor. It checks metadata rendering, math support, Chinese text, and the default code block API. It also checks structured citations and bibliography output such as **lamport1994latex**.

这是一份最小化回归示例，用于验证模板在中英混排场景下的基本可用性。

The template now provides branded section headings, structured author metadata, and a lightweight contribution model for team reports.

2 Math

The acceleration of gravity is written as

$$g = 9.81\,\mathrm{m/s^2}.$$

The template also provides semantic math helpers for vectors, matrices, norms, absolute values, and differentials:

$$\boldsymbol{v}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}, \tag{1}$$

$$\mathbf{K}\boldsymbol{x} = \boldsymbol{b}, \tag{2}$$

$$\|\boldsymbol{x}\|_2 \le |\alpha| + 2.5\,\mathrm{N}, \tag{3}$$

$$I = \int_0^T \mathrm{e}^{-t} \sin(\omega t)\ \mathrm{d}t. \tag{4}$$

In Chinese technical writing, we may also write:

$$\mathbf{A}^{\mathsf{T}}\boldsymbol{x} = \boldsymbol{y}, \quad F = ma.$$

3 Code

The inline API also supports `print("Hello SUSTech")` in running text.

Python

```python
def greet(name: str) -> None:
    print(f"Hello, {name}!")

greet("SUSTech")
```

Shell

```shell
$ xelatex example.tex
This is XeTeX...
Output written on example.pdf.
```

The structured interface supports formal references such as Code 1. See also Section 2, Equation (4), Table 1, and Figure 1. For bibliography-backed citations, see **shannon1948mathematical** and [**ctan-biblatex**].

Greeting Helper `src/greet.py`

```python
def greet(name: str) -> None:
    print(f"Hello, {name}!")


if __name__ == "__main__":
    greet("Engineering Report")
```

Code 1 Reference implementation for the greeting helper used in this report.

Build Log `terminal://xelatex`

```shell
$ xelatex example.tex
This is XeTeX, Version 3.141592653...
Output written on example.pdf (7 pages).
```

Code 2 Typical XeLaTeX build output for a local smoke test.

4 Code — Extended Languages and Advanced Blocks

4.1 Modern C++

The template ships a `[Modern]C++` language definition covering keywords from C++11 through C++23, including `auto`, `nullptr`, `constexpr`, `concept`, `co_await`, and STL type aliases.

moderncpp

```modern-cpp
#include <memory>
#include <concepts>
#include <print>

template<typename T>
concept Printable = requires(T x) { std::print("{}", x); };

auto make_greeting(Printable auto const& value) -> std::string {
    return std::format("Hello, {}!", value);
}

```

```cpp
int main() {
    auto msg = make_greeting(std::string{"SUSTech"});
    std::println("{}", msg);     // C++23 println
    auto ptr = std::make_unique<std::string>(msg);
    return nullptr != ptr ? 0 : 1;
}
```

4.2 MATLAB

The `[SUSTech]MATLAB` definition highlights control-flow keywords, built-in math functions, and plotting commands as a distinct token class.

matlab

```matlab
function results = run_experiment(data, alpha)
% Compute filtered statistics and plot
    n = numel(data);
    mu = mean(data);
    sigma = std(data);
    results.snr = mu / sigma;

    filtered = data(abs(data - mu) < alpha * sigma);
    figure;
    plot(filtered, 'b-o');
    xlabel('Sample Index');
    ylabel('Amplitude');
    title('Filtered Signal');
    grid on;
end
```

4.3 Enhanced YAML

The YAML definition now highlights boolean literals (`true`/`false`/`null`), anchor (`&`) and alias (`*`) markers, and block/flow scalars.

yaml

```yaml
build: &defaults
  enabled: true
  strict:  false
  timeout: null

production:
  <<: *defaults
  enabled: true
  replicas: 3
  env:
    DATABASE_URL: "postgres://db:5432/prod"
    DEBUG: false
```

4.4 Long Code Block with Pagination

`sustechlongcode` and `sustechlongshell` are breakable variants that print a "*(continued)*" header whenever the block spans a page boundary. The following block is intentionally long enough to exercise the mechanism in a dense document.

Signal Processing Pipeline src/pipeline.py

```python
"""
Signal processing pipeline for time-series sensor data.
Demonstrates sustechlongcode pagination in the SUSTech report template.
"""
from __future__ import annotations
import numpy as np
from dataclasses import dataclass, field
from typing import Sequence

@dataclass
class PipelineConfig:
    sample_rate: float = 1000.0          # Hz
    window_size: int   = 256
    overlap:     float = 0.5
    bands: list[tuple[float, float]] = field(
        default_factory=lambda: [(1, 10), (10, 50), (50, 200)]
    )

def bandpass(signal: np.ndarray, low: float, high: float,
             fs: float) -> np.ndarray:
    """Apply a simple FFT-based band-pass filter."""
    n    = len(signal)
    freq = np.fft.rfftfreq(n, d=1.0 / fs)
    spec = np.fft.rfft(signal)
    mask = (freq >= low) & (freq <= high)
    spec[~mask] = 0
    return np.fft.irfft(spec, n=n)

def windowed_rms(signal: np.ndarray, cfg: PipelineConfig) -> np.ndarray:
    step    = int(cfg.window_size * (1 - cfg.overlap))
    indices = range(0, len(signal) - cfg.window_size + 1, step)
    return np.array([
        np.sqrt(np.mean(signal[i : i + cfg.window_size] ** 2))
        for i in indices
    ])

def process(raw: Sequence[float], cfg: PipelineConfig | None = None
            ) -> dict[str, np.ndarray]:
    cfg    = cfg or PipelineConfig()
    arr    = np.asarray(raw, dtype=float)
    output = {}
    for low, high in cfg.bands:
        key         = f"band_{int(low)}_{int(high)}Hz"
        filtered    = bandpass(arr, low, high, cfg.sample_rate)
        output[key] = windowed_rms(filtered, cfg)
    return output

if __name__ == "__main__":
    rng  = np.random.default_rng(42)
    data = rng.standard_normal(4096)
    results = process(data)
    for name, rms in results.items():
        print(f"{name}: mean RMS = {rms.mean():.4f}")
```

4.5 Diff-Style Code Block

`sustechdiff` renders a unified diff inline; `sustechdiffblock` wraps it in a formal captioned block.

Diff

```diff
--- a/src/config.py
+++ b/src/config.py
@@ -1,8 +1,10 @@
 class Config:
-    DEBUG   = False
-    TIMEOUT = 30
+    DEBUG   = False           # unchanged
+    TIMEOUT = 60              # increased from 30
+    MAX_RETRIES = 3           # new field

-    def validate(self):
-        return self.TIMEOUT > 0
+    def validate(self) -> bool:
+        return self.TIMEOUT > 0 and self.MAX_RETRIES >= 1
```

Refactor: rename greet to hello `src/greet.py`

```diff
--- a/src/greet.py
+++ b/src/greet.py
@@ -1,6 +1,6 @@
-def greet(name: str) -> None:
-    print(f"Hello, {name}!")
+def hello(name: str) -> str:
+    return f"Hello, {name}!"

 if __name__ == "__main__":
-    greet("Engineering Report")
+    print(hello("Engineering Report"))
```

Code 3 Rename the public function and update its return type annotation.

5 Table

Item	Status
Template Core	Stable
Author Metadata	Implemented
Contribution Model	Structured

Module	Priority
Metadata	High
Visual System	High

Table 1 Styled table caption example

6 Figure

7 List

- SUSTech Green anchors headings, list markers, and header rules (VIS).
- Captions and lists are styled to match the template system.

1. Build the metadata layer.
2. Build the visual layer.

Figure 1 Styled figure caption example

(a) Baseline result

(b) Improved result

Figure 2 Two subfigures in one grouped figure

3. Validate with a real XeLaTeX compile.

8 Grouped Tables

Metric	Value
Accuracy	91%
Recall	88%

(a) Experiment A

Metric	Value
Accuracy	94%
Recall	90%

(b) Experiment B

Table 2 Two subtables in one grouped table

9 Diagram

10 Conclusion

The class should compile with XeLaTeX after the phase-one refactor. It should also support integrated bibliography output for books, papers, package pages, and code repositories such as [**pygments-github**].

第一阶段的目标是先完成模板内核重构，而不是一次性完成所有高级功能。

Figure 3 Wide figure placeholder for future two-column layouts

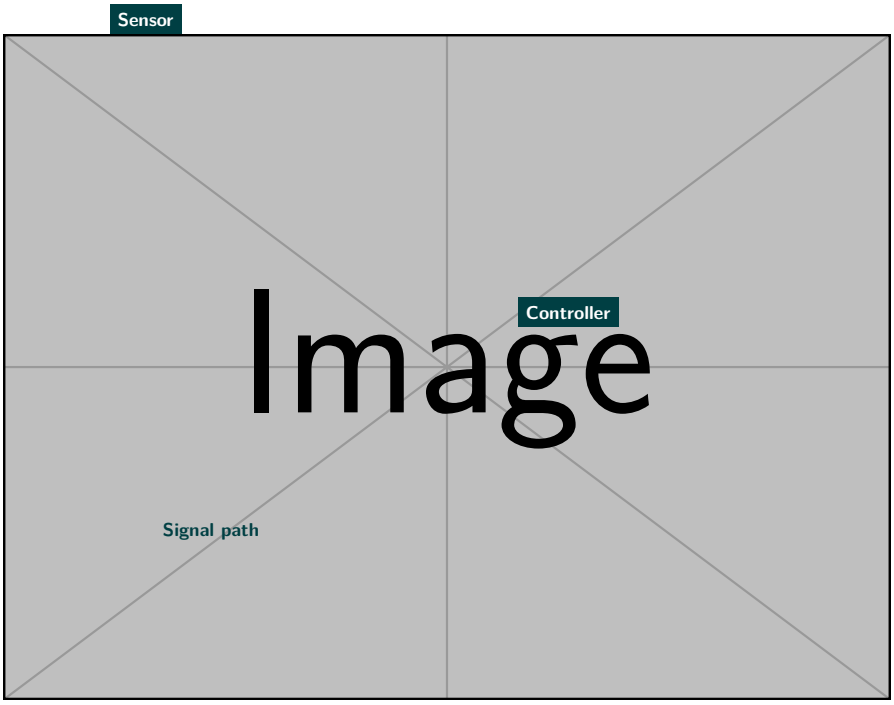

Figure 4 Annotated figure with branded overlay labels

Stage	Owner
Sampling	Alice
Validation	Bob
Writing	Wells

Reading Guide
This layout is useful when the table itself is compact, but the reader still needs a short explanation of column meaning, dataset scope, or evaluation conditions.

Table 3 Compact table with explanatory aside

Interpretation
The left panel can hold a plot, apparatus photo, or workflow snapshot. The right panel is reserved for engineering interpretation, assumptions, or key observations that are too dense for a short caption.

Figure 5 Side-by-side figure with explanatory aside

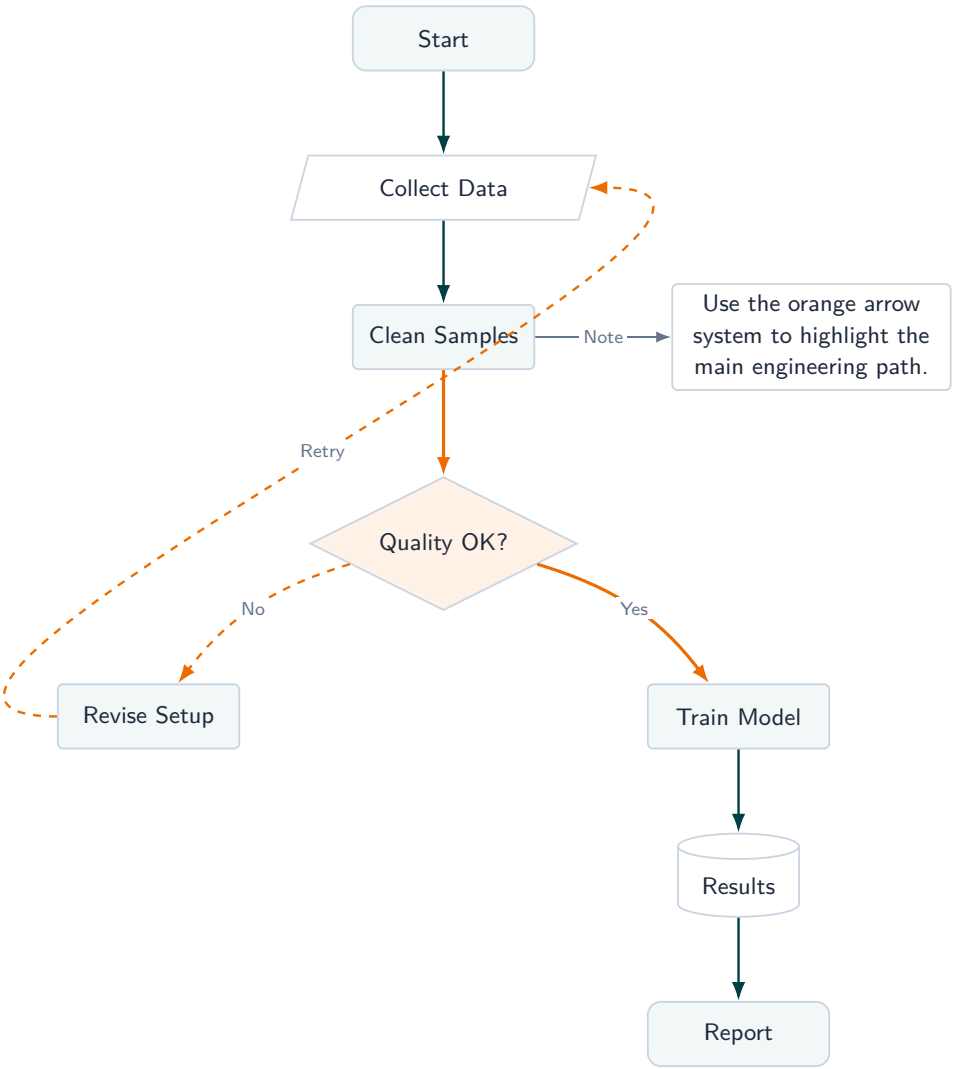

Figure 6 Branded engineering flowchart example

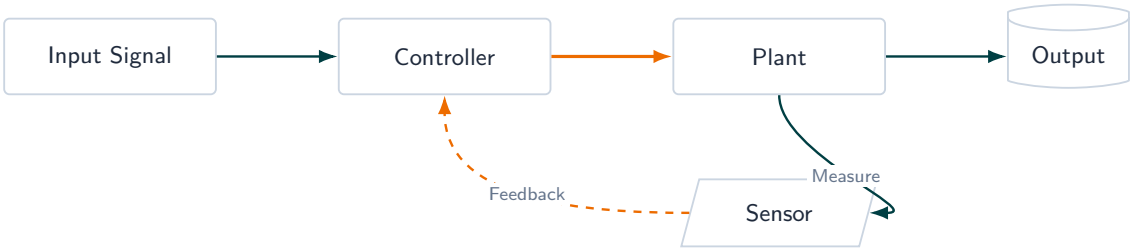

Figure 7 System block diagram template example

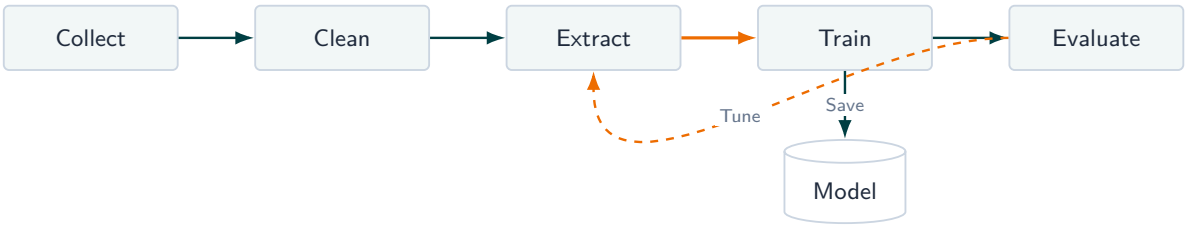

Figure 8 Data and experiment pipeline template example

Figure 9 Project timeline template example

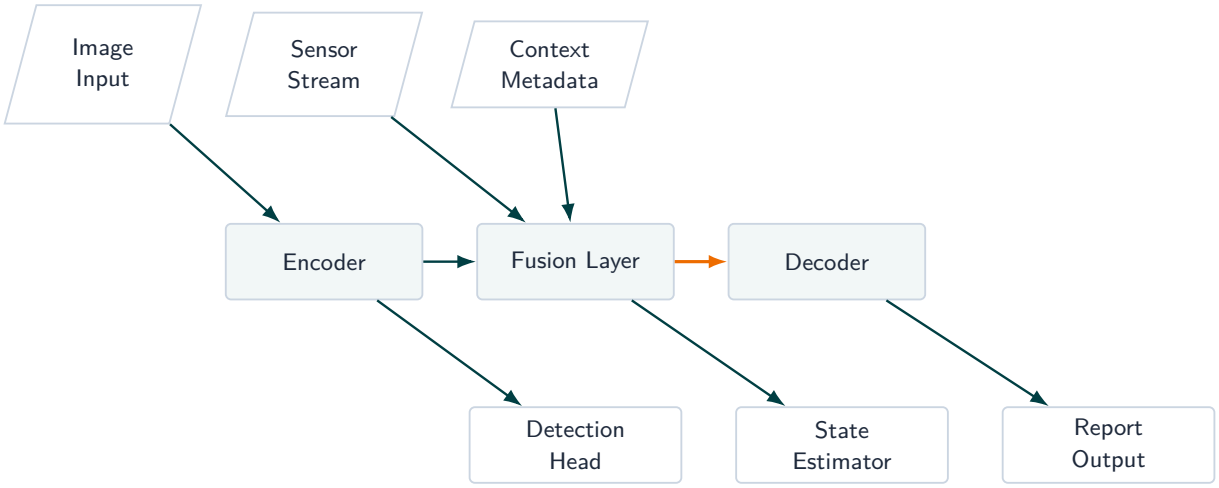

Figure 10 Method overview and model architecture template example

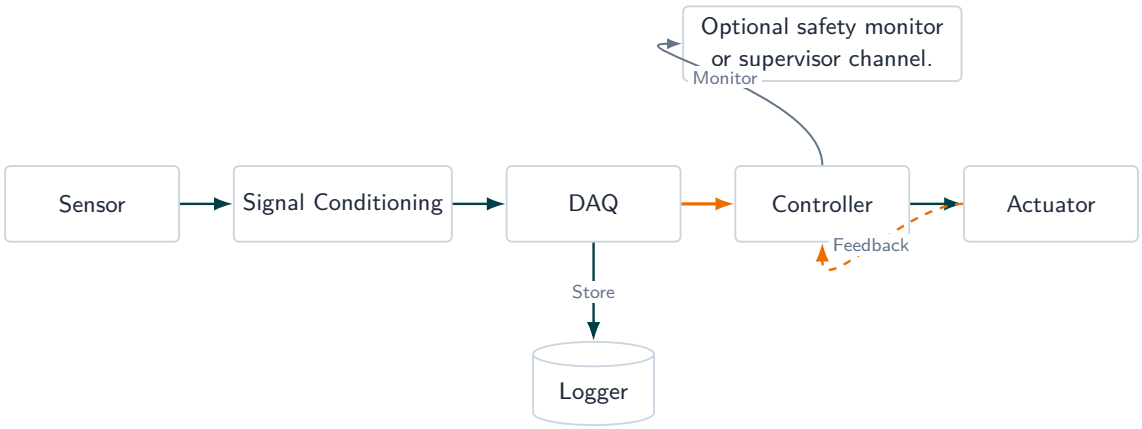

Figure 11 Experiment setup and acquisition chain template example